

Generative AI Analysis and Ethics Report

A Character-Level Transformer for Classical Arabic Poetry

Belal Nwiran

Overview

This project builds a small generative AI system that writes classical Arabic poetry. A poem here is a short list of lines (*bayts*). Each line has two half-lines (hemistichs), and the lines of a poem usually end with the same rhyme letter. The goal was to see how much of this form and style a small model can learn from examples, and where it fails.

I trained a character-level, decoder-only Transformer from scratch in PyTorch on about 29,700 poems from the Abbasid era. The poems come from the Ashaar dataset (Arabic Machine Learning, n.d.). A Transformer is a good fit for text generation, because its masked self-attention lets each position use all earlier positions to predict the next one (Vaswani et al., 2017). The authors of Ashaar also used a character-based GPT model for Arabic poetry generation (Alyafeai et al., 2023), which supports this choice. I used characters instead of words because the vocabulary is only 40 symbols, and because rhyme depends on the last letters of each line.

Training was stable and took 8.8 minutes. The validation loss fell from 3.84 to 1.776, which is a perplexity of about 5.9 per character. The generated poems follow the form well, but not the meaning. All 36 lines of the nine generated poems have the two-half format, and 7 of the 9 poems keep the same rhyme letter in all four lines. However, the poems also contain repeated words, invented words and broken rhymes. The main ethical concerns are that a few poets dominate the training text, that the dataset may only be used for research, and that generated poems could be mistaken for real classical poems.

The GitHub Repo: <https://github.com/bnwiran/generative-ai-applications-project>

Dataset or Prompt Description

The data comes from Ashaar, a collection of Arabic poems published on Hugging Face (Arabic Machine Learning, n.d.). It has two files with 127,315 poems each, so 254,630 poems in total, from 14 historical eras. According to the dataset page, it is released for research use only, it must not be used commercially, and poets can ask for their work to be removed (Arabic Machine Learning, n.d.). The poem links in the data point to aldiwan.net, so the poems appear to be collected from that website. I downloaded the files on October 3, 2026 (commit 9b5e723).

Because the dataset is large, I used one era only. The Abbasid era has 30,413 poems, which is the largest era after the modern era (55,137 poems). One era gives a consistent style, and the Abbasid poets lived at least seven centuries ago, while modern poems may still be protected by copyright.

Table 1 shows the cleaning steps. Each step removes poems that would harm training.

Table 1

Cleaning Steps for the Abbasid Poems

Step	Reason	Poems removed	Poems left
Keep the Abbasid era only (from 254,630 poems)	One consistent style; second-largest era	224,217	30,413
Remove duplicate poem links	Same poem twice (38 duplicate links in the full dataset, none in this era)	0	30,413
Remove empty poems	No text to learn from	1	30,412
Remove poems with an odd number of half-lines	The pairing of first and second halves is broken	390	30,022
Remove very long poems (more than 156 half-lines, the 99th percentile)	About 1% of the poems hold 12.6% of all characters	296	29,726
Remove poems with an empty half-line after text cleaning	The pairing would break	19	29,707

Note. The steps were applied in this order. The last step comes after the text cleaning described below.

After the first four steps, 29,726 poems from 634 poets remained, with 15.6 million characters. The text is concentrated in a few poets. The ten largest poets write 44.7% of the characters. The largest is al-Buḥturī (6.8%), followed by Ibn al-Rūmī (6.7%) and al-Sharīf al-Raḍī (6.3%).

The raw text had 74 different characters. I kept only the 36 Arabic letters and the space. I removed diacritics (marks for short vowels), because they were used inconsistently. I also removed tatweel, a stretching mark that made up 1,141,081 characters, or 7.3% of the text. All other characters (punctuation, digits, Latin letters and symbols) were about 1,900 characters in total and were replaced by spaces. I did not merge letter variants, so the spelling of the poems stays as written.

Each poem became one text string. The two half-lines of a line are separated by |, each line ends with a new line, and each poem ends with #. This gives a vocabulary of 40 symbols: 36 letters, the space and 3 format symbols. In the data, some words are cut between the two half-lines, so the separator is not always between two words. The final text has 11,557,072 characters.

I shuffled the poems with seed 42 and kept 90% for training (26,737 poems, 10,407,255 symbols) and 10% for validation (2,970 poems, 1,149,817 symbols). The split is by whole poem, so no poem is in both sets. The vocabulary was built from the training poems only, and every symbol in the validation set was in it.

Model Design and Training Approach

Architecture

The model is a decoder-only Transformer. It adds an embedding for each symbol and for each position (up to 256 positions), and then applies 6 Transformer blocks. Each block has 6 attention heads, a feed-forward layer of size 1,536 and residual connections, with layer normalization before each part. A final linear layer gives a score for each of the 40 symbols. The model has 10,776,576 parameters.

The attention is causal: a position can only look at earlier positions, which lets the model learn to predict the next symbol without seeing it (Vaswani et al., 2017). I checked this in the notebook: changing the symbols after position 100 did not change any earlier prediction. Before training, the loss was 3.717, close to 3.689, which is the loss of random guessing among 40 symbols. This is what a correct start should look like.

The sizes (6 layers, 6 heads, 384 dimensions and a context of 256) follow the character-level example in nanoGPT (Karpathy, 2022). They fit my data for three reasons. The model has about one parameter per training symbol (10.8 million parameters for 10.4 million symbols). It fits easily in the 12 GB memory of my GPU. And a context of 256 symbols covers a typical poem: the median poem has 176 symbols, and 75% of the poems have 358 or fewer. I used a dropout of 0.1 because the data is large for a model of this size.

Training

The loss is the cross-entropy between the predicted scores and the true next symbol, at every position of every block. Table 2 lists the settings. Every 250 steps I measured the loss on 50 random training batches and 50 random validation batches with dropout off, and I kept the model with the lowest validation loss.

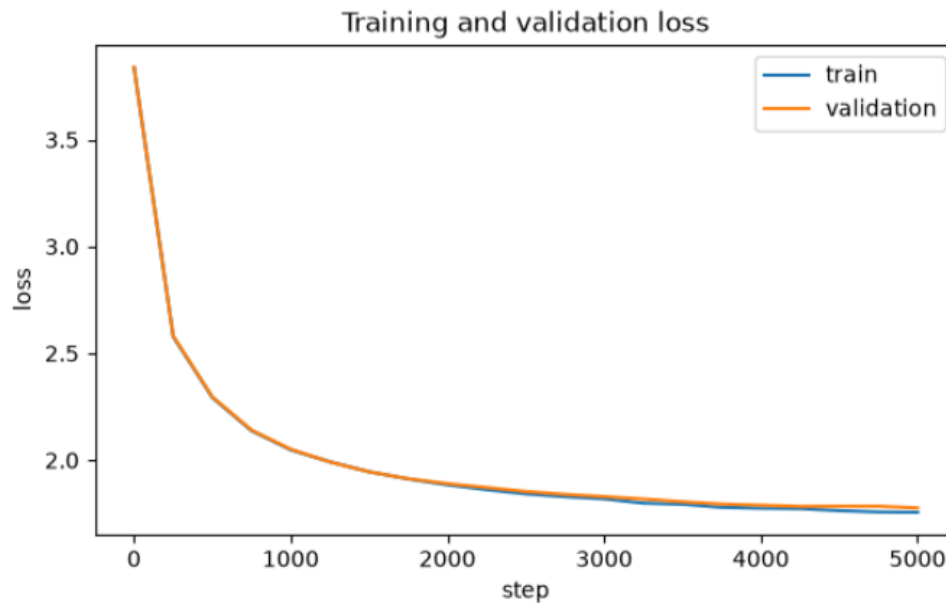
Table 2*Model and Training Settings*

Setting	Value
Parameters	10,776,576
Layers, heads, embedding size	6, 6, 384 (feed-forward size 1,536)
Context length	256 symbols
Dropout	0.1
Optimizer	AdamW (betas 0.9 and 0.95, weight decay 0.01)
Learning rate	3e-4, 200 warm-up steps, then cosine decay to 3e-5
Batch	64 random blocks of 256 symbols (16,384 symbols per step)
Steps	5,000 (about 8 passes over the training text)
Gradient clipping	1.0
Precision and hardware	bfloat16 mixed precision on an NVIDIA RTX 4070 (12 GB)
Random seed	42
Training time	8.8 minutes

Figure 1 shows the loss curves. The validation loss fell from 3.84 at step 0 to 1.776 at step 5,000, a perplexity of about 5.9 per character, while random guessing would give 40. Most of the improvement came in the first 1,000 steps (validation loss 2.05). The training and validation curves stay close together, 1.756 and 1.776 at the end, and the gap never exceeded 0.03, so the model did not overfit. The curves flatten as the learning rate falls: from step 4,000 to step 5,000 the validation loss only dropped from 1.789 to 1.776. This suggests that the model is limited by training time and size, and not by the amount of data. The best model was the one from step 5,000.

Figure 1

Training and Validation Loss



Note. The loss was measured every 250 steps on 50 random batches.

Generation

To generate a poem, the model starts from the end of a previous poem (# and a new line). It predicts a probability for each symbol, picks one symbol at random from the 20 most likely, and repeats this until the poem has the requested number of lines. Temperature divides the scores before they become probabilities, so low values give safer text and high values give more varied text. Choosing only the most likely symbols is known to give dull and repetitive text, and sampling from a limited set of likely symbols, such as top-k, was proposed as a better way (Holtzman et al., 2020). I blocked the end marker, so every poem has exactly the requested number of lines. The generation function can also take a start text, which the model continues.

External Examples and AI Assistance

This project is based on the general design of nanoGPT (Karpathy, 2022). I wrote the code in plain PyTorch with the help of an AI assistant (Claude, from Anthropic), which I used for planning, drafting and reviewing code and text. I ran all the code, checked the results, and made the final decisions. The model was trained from scratch, without pretrained weights.

Output Evaluation and Interpretation

I generated nine poems of four lines, three at each temperature (0.5, 0.8 and 1.0), and one poem from a given start (*yā layl*, "O night"). The same three seeds were used at every temperature. I read the poems as an Arabic reader and also counted the line format and the

rhyme. I did not use a reference-based metric such as BLEU, because free generation has no reference poem. The loss and perplexity above show how well the model predicts real text, but earlier work on text generation also looks at other signs of quality, such as repetition and human judgment (Holtzman et al., 2020).

Strengths

- **Format.** All 36 lines have exactly one separator, so the model learned the two-half structure.
- **Rhyme.** In 7 of the 9 poems, all four lines end with the same letter. For example, one poem at temperature 0.5 ends its four lines with the sound *-ād* (Table 3).
- **Word splits.** The model copies the habit of the data to cut a word between the two halves of a line (Table 3).
- **Classical vocabulary.** The poems use words and openings that are common in the training poems, such as *al-dahr* ("time, fate") and *al-mulk* ("the kingdom").
- **Start text.** Given *yā layl*, the model continued the word correctly as *yā laylah* and went on in the same format.

Failure cases

- **No clear meaning.** The phrases are fluent one at a time, but the lines do not form one thought.
- **Repetition.** In sample 2 at temperature 0.5, *al-mulk* ends lines 1 and 3, and line 2 repeats *al-ayyām* ("the days") twice. In sample 1 at temperature 0.8, the same word ends lines 1 and 3. In the poem from the start *yā layl*, line 1 ends both halves with the same word.
- **Invented words and spelling errors.** Several words do not exist in Arabic, and some words are misspelled (Table 3).
- **Broken rhyme.** In 2 of the 9 poems, one line breaks the rhyme of the other three (Table 3).

Table 3 shows examples.

Table 3

Examples From the Generated Poems

Behavior	Arabic	Transliteration	Note
Rhyme works	العباد، السماد، البلاد، الأيد	al-‘ibād, al-samād, al-bilād, al-ayād	The four line endings all end in <i>-ād</i> (temperature 0.5, sample 3).
Rhyme over meaning	السماح في السماد	al-samāḥ fi al-samād	"Generosity in the fertilizer." The rhyme word fits the sound but not the meaning (temperature 0.5, sample 3, line 2).
Word split	من الده ر	min al-dah r	<i>al-dahr</i> is cut between the two halves, as in many training lines (temperature 0.5, sample 1, line 4).
Repetition	الملك	al-mulk	Ends lines 1 and 3 (temperature 0.5, sample 2).
Invented words	الولك لتلومس جنانبه	al-walak, li-talūmas, jananibih	Not found as Arabic words (temperature 0.5 sample 2, and temperature 0.8 samples 1 and 2).
Spelling error	أرجوا	arjū + extra alif	An extra alif after the final waw (temperature 1.0, sample 3).
Broken rhyme	ترجع، المعنى، المنيع، السمع	tarji‘, al-ma‘nā, al-manī‘, al-sam‘	Line 2 ends in <i>-ā</i> , the others in <i>-‘</i> (temperature 0.8, sample 3).

Effect of temperature

Repetition appeared at temperatures 0.5 and 0.8, and invented words and spelling errors were mostly at 0.8 and 1.0. The rhyme was complete in 3 of 3 poems at 0.5, and in 2 of 3 poems at both 0.8 and 1.0. With only three poems per temperature, these are impressions and not trends. Also, the same seeds were used at every temperature, so the poems are not independent. For example, sample 2 starts with the same phrase (*tanāza‘ al-mulk*) at all three temperatures.

Interpretation

The results show that the model learned the form of classical poetry (line structure, rhyme and vocabulary) better than its meaning. This is consistent with the perplexity of 5.9: the model is still often unsure about the next letter. A larger model or longer training may improve the meaning, but I did not test this.

Ethical Considerations and Responsible Use

Bias and representation

The training text is not balanced. Ten poets write 44.7% of the text, and the largest, al-Buḥturī, writes 6.8%. The other 624 poets share the rest. I kept this natural distribution, so the style the model learns is mostly the style of a few well-known poets. The dataset also reflects what one website chose to publish, and it covers only one era, so other eras are not represented. Bender et al. (2021) explain that models trained on collected data repeat the views and gaps of that data, and they recommend documenting the data carefully. For this reason, the notebook documents the source, the license and the poet concentration.

Creative ownership

The Abbasid poets lived at least seven centuries ago, so their poems are far older than any copyright term. Still, the dataset is a digital collection gathered from a website, and its license limits it to research use, forbids commercial use and lets poets ask for removal (Arabic Machine Learning, n.d.). I followed these terms: the project is academic, and the repository contains only the cleaned subset, under the same terms.

A second concern is copying. A model can repeat lines from its training text, and presenting such lines as new work would be unfair to the poets. I did not test whether the generated lines appear in the training poems, so this is a risk and not a finding. The model has no poet names in its text, so it cannot be asked to write in the style of a named poet. However, the style it learned is mostly that of the most frequent poets.

Misuse

The generated poems imitate classical vocabulary and form, so readers who are not experts may take them for real classical poetry. Bender et al. (2021) warn that fluent text from a language model can make readers see meaning and authority that are not there. In my samples, a careful Arabic reader can see the problems: invented words, spelling errors and lines without clear meaning. But the poems could still be shared as authentic or attributed to real poets, which would spread false information about Arabic heritage.

Responsible use in this project

Several choices support responsible use. I used only old poems, I did not put poet names in the training text, I documented the data source and license, and I report the failure cases openly. I also suggest three rules for anyone using the model: label generated poems as machine-generated, never attribute them to a real poet, and do not sell them, in line with the dataset license. These steps reduce the risk but do not remove it.

Limitations and Future Improvements

Limitations

- **Small model and short training.** The model has 10.8 million parameters and trained for 8.8 minutes. The validation loss was still falling slowly at the end.
- **Meaning and word quality.** The poems contain invented words, repetition and broken rhymes, and the lines have no clear meaning.
- **Meter is not modeled.** The dataset gives a meter label for each poem, but I did not use it, and I did not test whether the generated lines follow a classical meter.
- **Small evaluation.** I judged nine poems and one extra poem from one training run (seed 42), read by one person. The temperature comparison is only an impression, and I did not test copying.
- **Data.** The data covers one era, is concentrated in a few poets and was collected from one website. Some words are cut between half-lines in the data, and the model copies this. The validation poems come from the same poets and era, so the validation loss shows fit to the same kind of data, not to new poets.

Future improvements

1. Train longer or with a larger model (for example 15,000 steps) and compare the samples. This is cheap and may improve meaning.
2. Test copying by searching the generated lines in the training text, and measure rhyme on more poems (for example 30 per temperature), compared with real poems.
3. Use the meter label and the rhyme letter as conditions for the model, as in conditional poetry generation (Alyafeai et al., 2023).
4. Limit the share of each poet in the training text, to make the representation more balanced.
5. Ask several Arabic readers to rate the poems, to replace the judgment of one reader.

References

- Alyafeai, Z., Al-Shaibani, M. S., & Ahmed, M. (2023). *Ashaar: Automatic analysis and generation of Arabic poetry using deep learning approaches* (arXiv:2307.06218). arXiv.
<https://doi.org/10.48550/arXiv.2307.06218>
- Arabic Machine Learning. (n.d.). *Ashaar* [Data set]. Hugging Face. Retrieved October 3, 2026, from <https://huggingface.co/datasets/arbml/ashaar>
- Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency* (pp. 610–623). Association for Computing Machinery. <https://doi.org/10.1145/3442188.3445922>
- Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. In *International Conference on Learning Representations*.
<https://arxiv.org/abs/1904.09751>
- Karpathy, A. (2022). *nanoGPT* [Computer software]. GitHub.
<https://github.com/karpathy/nanoGPT>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems* (Vol. 30, pp. 5998–6008). Curran Associates.